Lab 21: Inheritance and Polymorphism

- Due Wednesday by 11:59pm
- Points 5
- Submitting an external tool
- Available Jul 20 at 6pm - Jul 22 at 11:59pm

Learning Objectives

- Build inheritance hierarchy.
- Use inherited methods and attributes.

Completion Requirements

- Submit Lab21.py

Q1: Base Class - ElectronicDevice

Write a class `ElectronicDevice` that serves as the foundation for an inventory system. It should contain the following attributes and instance methods:

Attributes

1. `brand`: A string representing the manufacturer of the device.
2. `model`: A string representing the specific model name.
3. `price`: A float representing the cost of the device.

Methods

1. **__init__(self, brand, model, price)**: Initializes the `brand`, `model`, and `price` attributes.
2. **apply_discount(self, percentage)**: Reduces the `price` attribute by the given percentage value (e.g., passing 10 reduces the price by 10%).
3. **__str__(self)**: Returns a clean string configuration containing the device info formatted as: "Brand Model - $Price" (with the price rounded to 2 decimal places). See example.
4. **__lt__(self, other)**: Compares the `price` of the current device against another device object, returning True if it is strictly cheaper, and False otherwise.

Example 1:

```
>> device = ElectronicDevice("Generic", "Gadget", 100.00)
>> device.apply_discount(15)
>> print(device)
Generic Gadget - $85.00
```

Q2: Derived Class - SmartPhone

Create a child class named `SmartPhone` that inherits directly from `ElectronicDevice`. It should feature the following unique configurations:

Attributes

1. `storage_gb`: An integer value denoting total internal hardware capacity.

Methods

1. **__init__(self, brand, model, price, storage_gb)**: Accepts four parameters. You must call `super().__init__()` to initialize the parent elements before binding the unique storage attribute.
2. **install_app(self, app_name)**: Returns a string confirming execution status matching: "Installing [app_name] on [brand] [model]..."

Example 2:

```
>> phone = SmartPhone("Google", "Pixel 8", 699.99, 128)
>> phone.install_app("Mobile Python IDE")
'Installing Mobile Python IDE on Google Pixel 8...'
>> phone.apply_discount(10)
>> print(phone)
Google Pixel 8 - $629.99
```

Q3: Derived Class - Laptop

Create an additional separate child class named `Laptop` that inherits directly from `ElectronicDevice`. It should feature the following unique configurations:

Attributes

1. `ram_gb`: An integer value denoting total system working memory.

Methods

1. **__init__(self, brand, model, price, ram_gb)**: Accepts four parameters. You must use `super().__init__()` to route parameters to the parent class correctly.
2. **upgrade_ram(self, additional_ram)**: Modifies the `ram_gb` count upward by the designated integer amount and returns a string confirmation matching: "[brand] [model] upgraded to

[ram_gb]GB RAM."

Example 3 (Polymorphic Comparisons):

```
>> laptop = Laptop(“Dell”, “XPS 13”, 1199.99, 16)
>> laptop.upgrade_ram(16)
'Dell XPS 13 upgraded to 32GB RAM.'
>> phone = SmartPhone(“Google”, “Pixel 8”, 629.99, 128)
>> phone < laptop
True
```

This tool needs to be loaded in a new browser window

Load Lab 21: Inheritance and Polymorphism in a new window